

AIRFLOW NOTES



hguneth!



- airflow ignore → to ignore folders in dags (like gitignore)

- with clause → permette di non passare dag=dag ad ogni task

dag with same id → non abbiamo errori ma non sappiamo quale viene caricato

description → buona convenzione aggiungere dettagli

start_date → necessaria per i task, non per il dag → buona nome id dag

schedule_interval → ogni quanto viene lanciato

dagrun_timeout → tempo massimo di esecuzione di un dag → NO DEFAULT

tags → si può usare per filtrare i dag

catchup → set always to false

max_active_runs → numero massimo di dag in esecuzione

Utility

DAG

ARGS

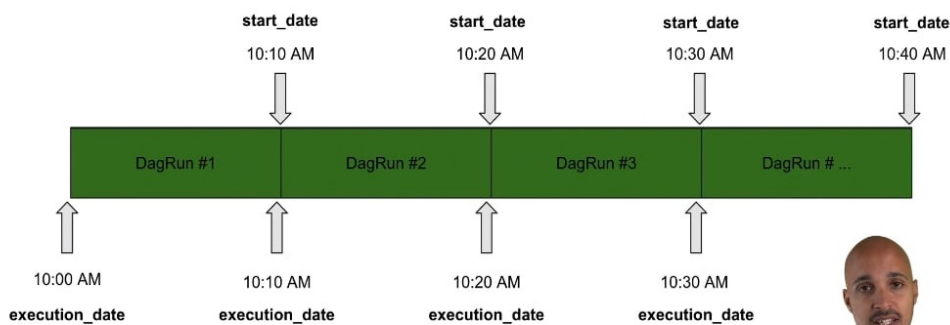
SCHEDULE

CRON

VS

TIMEDelta

Assuming a start date at 10:00 AM and a schedule interval every 10 mins



DAG : processor_customer ← start_date : 01/01 10:00 AM
schedule_interval = "@daily" → 00***

① 01/01 00:00 ② 01/02 00:00 ③

schedule_interval = timedelta(days=1)

① 01/01 10:00 AM ② 01/02 10:00 AM

We can use **Variable** to share common variable between nodes.
Esse possono essere integrate e nascoste, quindi anche per dati sensibili (also good)

Some Additional Notes

1: There are 6 different ways of creating variables in Airflow 🤖

- Airflow UI 📄
- Airflow CLI 📄
- REST API 📄
- Environment Variables ❤️
- Secret Backend ❤️
- Programmatically 🤖

Whichever one you choose depends on both use case and personal preference.

2: By creating a variable with an environment variable you:

- avoid making a connection to your DB
- hide sensitive values (the variable can only be fetched within a DAG)

3: It's worth noting that it is possible to create connections with environment variables. You just have to export the env variable...

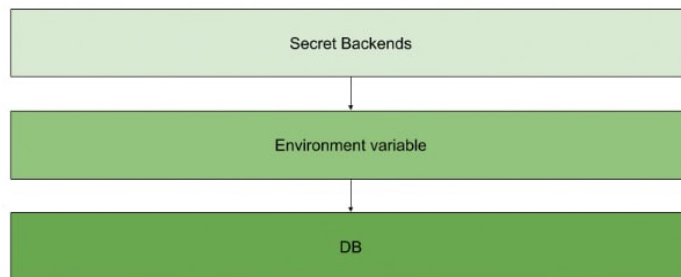
```
AIRFLOW_CONN_NAME_OF_CONNECTION=your_connection
```

...and get the same benefits as stated above.

4: Ultimately, if you really want a secure way of storing your variables or connections, use a Secret Backend.

To learn more about this, click on this [link](#)

5: Finally, here is the order in which Airflow checks if your variable/connection exists:



Another way to share variable between tasks is XCOM:

- `xcom_push(task_id, value)`
- `xcom_pull(task_id)`

In `pythonOperator` or other with **RETURN**, the return is automatically pushed.

✓
A
R
I
Q
B
L
E

DECORATORS → Si possono usare per evitare di creare i task con gli operatori. Si possono creare degli dependencies anche tramite l'uso di funzioni.

TASK GROUP → Si può usare per raggruppare più task in un subdag

BRANCH OPERATOR → Permette di selezionare quali task eseguire in base a condizioni

TRIGGER RULE → Indica le condizioni per cui un task viene eseguito (attolla immediati) es. all failed → cancel

CROSS-DOWNSTREAM → Dipendenza tra liste di task

CHAIN → Sequenza di task o downstream task

CONCURRENCY → Number task in execution

TASK CONCURRENCY → Number of task in execution

POOL → Definisce le mode di esecuzione del task e per gestire le priorità

DEPEND_ON_PAST → Permette di dire ad un task che dipende dal passato, esecuzioni passate separate

WAIT_FOR_DOWNSTREAM → Esegue solamente se lo stesso task è in success nel precedente

ON_SUCCESS_CALLBACK → Esegue una funzione se non c'è stato errore

ON_FAILURE_CALLBACK → Esegue una funzione se c'è stato errore

ON_RETRY_CALLBACK → Esegue una funzione quando ritenta l'esecuzione

RETRY → Numero delle volte che il task viene rieseguito

RETRY_DELAY → Dopo quanto riprovare

RETRY_EXPONENTIAL_BACKOFF → Crescita in modo esponenziale dell'attesa

MAX_RETRY_DELAY → Massimo tempo d'attesa

SLA → Dopo un tempo prefissato indica lo stato dei task (a livello di dag)

<https://academy.astronomer.io/astronomer-certification-apache-airflow-dag-authoring-preparation/899677>

EXTERNAL TASK ID → Aspetta task di altri dag (usato in runtime)

Parametri

Gestione
Fallimenti

DAG Dinamici